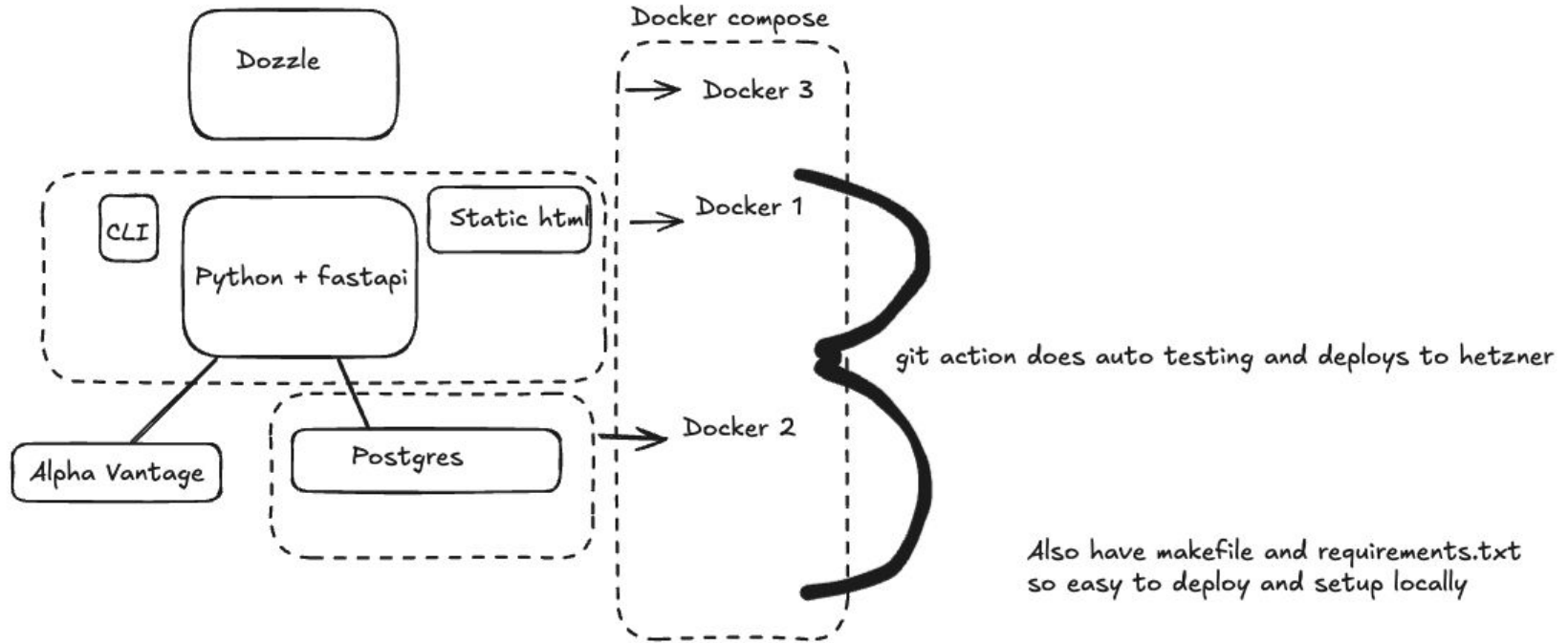
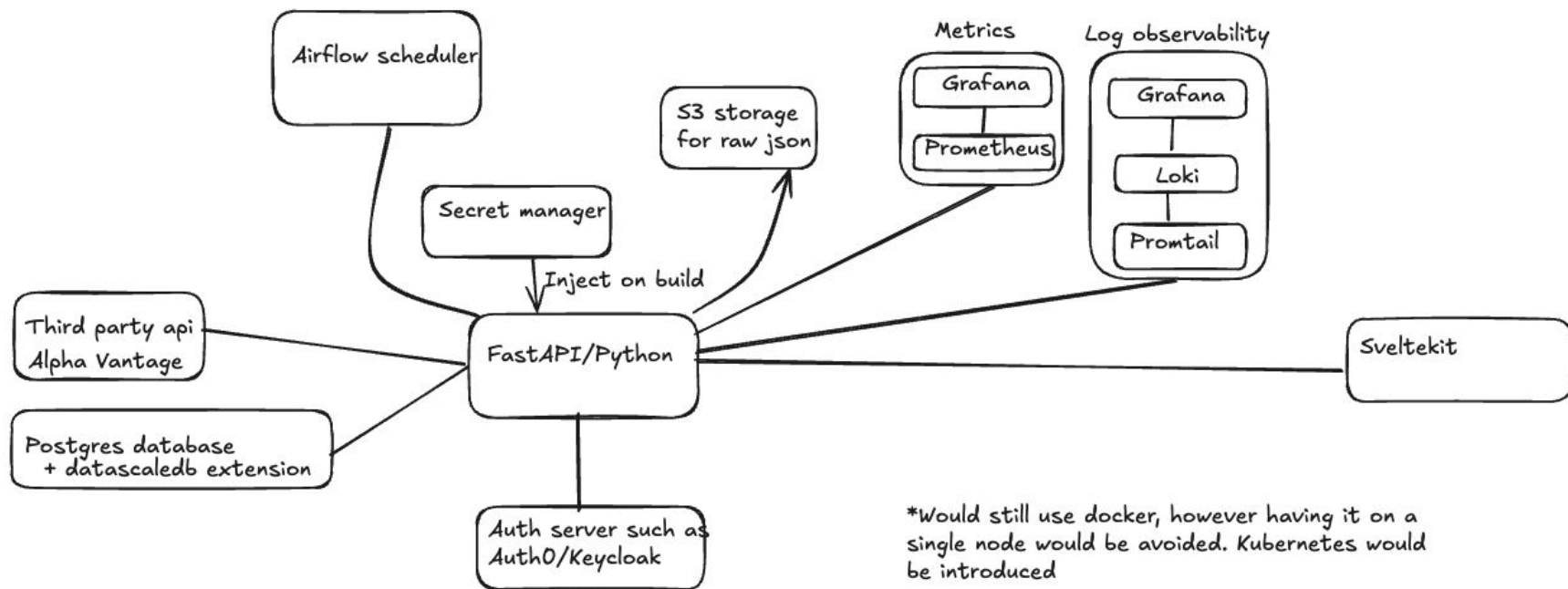


Oakland interview solution walkthrough

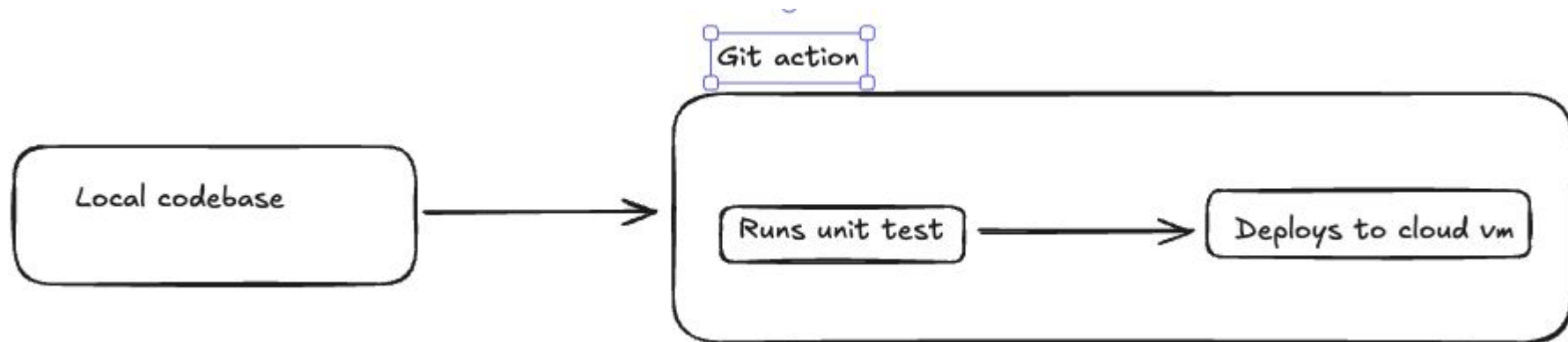
Architecture, where we are at



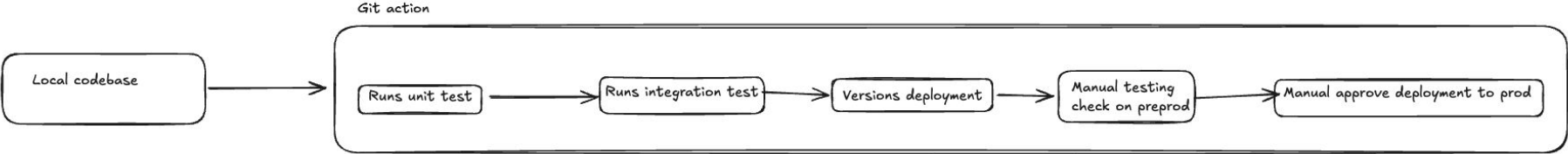
Architecture, where it would go to



CI/CD where it is at



CI/CD where it could be



Harness/
ArgoCD

Schema

<http://localhost:8000/data-viewer>

Star schema silver layer

Made a few assumptions. Mostly this is a write light, read heavy application

Silver

bronze_api_responses

```
CREATE TABLE IF NOT EXISTS bronze_api_responses (  
  id SERIAL PRIMARY KEY,  
  api_called VARCHAR(50) NOT NULL,  
  ticker VARCHAR(10) NOT NULL,  
  status VARCHAR(20) NOT NULL,  
  response_json JSONB NOT NULL,  
  extracted_at TIMESTAMPTZ WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

fact_daily_prices

```
CREATE TABLE IF NOT EXISTS fact_daily_prices (  
  stock_id INT REFERENCES dim_stocks(id) ON DELETE CASCADE,  
  price_date DATE NOT NULL,  
  open_price NUMERIC(12, 4) NOT NULL,  
  high_price NUMERIC(12, 4) NOT NULL,  
  low_price NUMERIC(12, 4) NOT NULL,  
  close_price NUMERIC(12, 4) NOT NULL,  
  volume BIGINT NOT NULL,  
  PRIMARY KEY (stock_id, price_date)  
);
```

dim_stocks

```
CREATE TABLE IF NOT EXISTS dim_stocks (  
  id SERIAL PRIMARY KEY,  
  ticker VARCHAR(10) UNIQUE NOT NULL,  
  company_name VARCHAR(255) NOT NULL,  
  should_fetch BOOLEAN DEFAULT TRUE,  
  last_price_sync_at TIMESTAMPTZ WITH TIME ZONE,  
  last_balance_sheet_sync_at TIMESTAMPTZ WITH TIME ZONE,  
  created_at TIMESTAMPTZ WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

fact_balance_sheets

```
CREATE TABLE IF NOT EXISTS fact_balance_sheets (  
  stock_id INT REFERENCES dim_stocks(id) ON DELETE CASCADE,  
  fiscal_date_ending DATE NOT NULL,  
  period_type VARCHAR(20) NOT NULL,  
  reported_currency VARCHAR(10),  
  total_assets BIGINT,  
  total_current_assets BIGINT,  
  cash_and_cash_equivalents BIGINT,  
  cash_and_cash_equivalents_investments BIGINT,  
  inventory BIGINT,  
  current_net_receivables BIGINT,  
  total_non_current_assets BIGINT,  
  property_plant_and_equipment BIGINT,  
  accumulated_depreciation BIGINT,  
  intangible_assets BIGINT,  
  goodwill BIGINT,  
  short_term_investments BIGINT,  
  total_liabilities BIGINT,  
  total_current_liabilities BIGINT,  
  current_accounts_payable BIGINT,  
  short_term_debt BIGINT,  
  total_non_current_liabilities BIGINT,  
  long_term_debt BIGINT,  
  total_shareholder_equity BIGINT,  
  retained_earnings BIGINT,  
  common_stock BIGINT,  
  shares_outstanding BIGINT,  
  PRIMARY KEY (stock_id, fiscal_date_ending, period_type)  
);
```

gold_latest_daily_prices

```
CREATE MATERIALIZED VIEW gold_latest_daily_prices AS  
WITH ranked_prices AS (  
  SELECT  
    s.ticker,  
    s.company_name,  
    p.price_date,  
    p.open_price,  
    p.high_price,  
    p.low_price,  
    p.close_price,  
    p.volume,  
    ROW_NUMBER() OVER (PARTITION BY p.stock_id ORDER BY p.price_date DESC) as rn  
  FROM fact_daily_prices p  
  JOIN dim_stocks s ON p.stock_id = s.id  
)  
SELECT  
  ticker,  
  company_name,  
  price_date,  
  open_price,  
  high_price,  
  low_price,  
  close_price,  
  volume  
FROM ranked_prices  
WHERE rn = 1;
```

MATERIALIZED VIEW gold_latest_daily_prices

Strengths

Scalable I/O & Performance Architecture

- FastAPI with async/await allows non blocking operations. Performance is high according to benchmarks
- Connection pooling to prevent database saturation
- Medallion architecture including materialised view for fast read times
- Postgres can be expanded using timescaleDB or horizontally scaled for better performance and scalability

```
async with httpx.AsyncClient() as client:  
    response = await client.get(self.base_ur
```

```
async def init_db_pool() -> None:  
    global _pool  
    if _pool is None:  
        _pool = await asyncpg.create_pool(  
            os.getenv("DATABASE_URL"),  
            min_size=1, max_size=10  
        )  
        logger.info("Database connection pool initialized.")  
  
async def close_db_pool() -> None:  
    global _pool  
    if _pool:  
        await _pool.close()  
        _pool = None  
        logger.info("Database connection pool closed.")  
  
def get_pool() -> asyncpg.Pool:  
    if _pool is None:  
        raise RuntimeError("Database pool is not initialized")  
    return _pool
```


Portable & Cloud-Agnostic Deployment

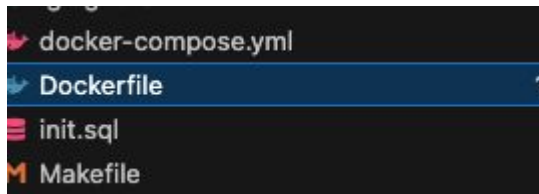
- Deployed using docker with CI/CD automation
- Can easily migrate to a new cloud VM with no proprietary requirements
- Makefile makes it easy
- No hardcoded values, .env easy to update
- Less 'works on my machine' problems

```
# API Credentials
ALPHA_VANTAGE_KEY=your_alpha_vantage_key_here
# Testing Toggles
USE MOCK_API=true

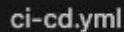
# Database Configuration
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
POSTGRES_DB=my_stock_db
DATABASE_URL=postgresql://postgres:postgres@postgres:5432/my_stock_db

DEADMANS_SWITCH_URL=https://hc-ping.com/your-unique-uuid-here

# Feature Flags
ENABLE_CLI_SYNC=true
```



```
docker-compose.yml
Dockerfile
init.sql
Makefile
```



```
ci-cd.yml
```

Data Reliability & Strict Input Contracts

- Pydantic models enforce strict data validation
- JSON schema validation catches malformed responses
- Type hints enable static type checking
- Database constraints prevent invalid states

```
ticker VARCHAR(10) UNIQUE NOT NULL,  
open_price VARCHAR(255) NOT NULL
```

```
class AlphaVantagePriceParser(BaseModel):  
    records: List[DailyPriceRecord]  
  
    @model_validator(mode="before")  
    @classmethod  
    def transform_raw_data(cls, data: dict) -> dict:  
        time_series = data.get("Time Series (Daily)", {})  
        parsed_records = []  
        for date_str, metrics in time_series.items():  
            parsed_records.append({  
                "price_date": date_str,  
                "open_price": metrics.get("1. open"),  
                "high_price": metrics.get("2. high"),  
                "low_price": metrics.get("3. low"),  
                "close_price": metrics.get("4. close"),  
                "volume": metrics.get("5. volume")  
            })  
        return {"records": parsed_records}
```

```
class DailyPriceRecord(BaseModel):  
    price_date: date  
    open_price: Decimal  
    high_price: Decimal  
    low_price: Decimal  
    close_price: Decimal  
    volume: int  
  
    @model_validator(mode="after")  
    def validate_price_bounds(self) -> "DailyPriceRecord":  
        if any(p <= 0 for p in [self.open_price, self.high_price, self.low_price, self.close_price]):  
            raise ValueError("Data QA Alert: Trading prices must be strictly positive.")  
        if self.volume < 0:  
            raise ValueError("Data QA Alert: Traded volume cannot be negative.")  
  
        if self.low_price > self.high_price:  
            raise ValueError("Data QA Alert: Low trading bounds cannot exceed high boundaries.")  
        return self
```

Decoupled, Modular Architecture

- Any part can be improved/switched out relatively easily
- For flexibility reasons, this is a great advantage. No true requirements given so flexibility is essential to quickly iterate if needed.

Development Velocity via Mock Data

- Local mock data server returns real responses
- Toggle between mock and live API via configuration without code changes
- Scenarios can be manually crafted for edge-case testing
- Avoids API rate limits during development

Using factory pattern so interchangeable with out code knowing

```
@abstractmethod
async def fetch_stock_data(self, ticker: str) -> dict:
    pass

@abstractmethod
async def fetch_balance_sheet_data(self, ticker: str) -> dict:
    pass

class RealAlphaVantageClient(StockApiClient):
    def __init__(self):
        self.api_key = os.getenv("ALPHA_VANTAGE_KEY")
        self.base_url = "https://www.alphavantage.co/query"

    async def fetch_stock_data(self, ticker: str) -> dict:
        params = {
            "function": "TIME_SERIES_DAILY",
            "symbol": ticker.upper(),
            "apikey": self.api_key
        }
        async with httpx.AsyncClient() as client:
            response = await client.get(self.base_url, params=params)
            response.raise_for_status()
            return response.json()

    async def fetch_balance_sheet_data(self, ticker: str) -> dict:
        params = {
            "function": "BALANCE_SHEET",
            "symbol": ticker.upper(),
            "apikey": self.api_key
        }
        async with httpx.AsyncClient() as client:
            response = await client.get(self.base_url, params=params)
            response.raise_for_status()
            return response.json()

class MockAlphaVantageClient(StockApiClient):
    def __init__(self):
        self.current_dir = os.path.dirname(os.path.abspath(__file__))

    async def fetch_stock_data(self, ticker: str) -> dict:
        normalized_ticker = ticker.upper()
        ticker_file_map = {
            "AAPL": "apple_daily.json",
            "IBM": "ibm_daily.json"
        }
        filename = ticker_file_map.get(normalized_ticker, "apple_daily.json")
        file_path = os.path.join(self.current_dir, "mock_data", filename)
        return self._load_json_file(file_path, normalized_ticker)

    async def fetch_balance_sheet_data(self, ticker: str) -> dict:
        normalized_ticker = ticker.upper()
        ticker_file_map = {
            "IBM": "ibm_balance.json",
            "AAPL": "apple_balance.json",
        }
        filename = ticker_file_map.get(normalized_ticker, "ibm_balance.json")
        file_path = os.path.join(self.current_dir, "mock_data", filename)
```

MS 2 OUTPUT TERMINAL PORTS

What could be improved

Robust Pipeline Orchestration

Current

- No retry logic for failed API calls
- No task history or audit trail
- Single node; no redundancy
- No visibility into task state
- Cron is internal

How to fix

- Apache Airflow or Dagster for external orchestration
- Automatic retries with exponential backoff
- Multi-machine scheduling for parallel tasks

Dedicated Object Storage for Bronze Layer

Current

- Raw API responses stored in PostgreSQL
- Expensive storage for historical data
- Mixing OLTP and OLAP in one database

How to fix

- Migrate to cloud object storage (AWS S3, GCS)
- Partition by date for efficient querying
- Archive old data to cheaper storage tiers

Database Migration Management

Current

- Schema initialised via static `init.sql`
- Manual SQL edits for changes
- No version control of schema
- No rollback capability

How to fix

- Alembic or Flyway for automated migrations
- Version-controlled schema changes
- Zero-downtime deployments
- Audit trail of all changes

Dead Letter Queue (DLQ) for Failed Validations

Current

- Invalid payloads logged then discarded
- No way to inspect or replay failed data
- Manual data reconstruction required

How to fix

- Dedicated DLQ table for validation failures
- Dashboard for reviewing failed records
- Add cases to testing suite

```
except (ValidationError, ValueError) as row_err:
    corrupted_records_count += 1
    # In production, you would write this row into a dead_letter_queue table here.
    logger.warning(f"[DLQ][{ticker}] Isolated a corrupt data row on date {date_str}: {str(row_err)}")
    continue
```

Enterprise Observability & Security

Current

- Logs to stdout; aggregated locally via Dozzle
- No authentication on API routes
- No encryption for API keys
- No audit logging

How to fix

- Grafana Loki + Promtail for logging
- Prometheus + Grafana for metrics
- JWT token authentication
- RBAC with admin/read-only roles
- Secrets manager for API keys
- TLS 1.3 for all communication

Websocket logic

Current

- Has no logic looking for data mutation etc, simply sends every 1.5 seconds
- Minimal advantage as is over polling

How to fix

- Only resend data on data mutation

Websocket logic

Current

- Has no logic looking for data mutation etc, simply sends every 1.5 seconds
- Minimal advantage as is over polling

How to fix

- Only resend data on data mutation